

An Efficient Submesh Allocation Strategy for Mesh Computer Systems

Po-Jen Chuang and Nian-Feng Tzeng

The Center for Advanced Computer Studies
University of Southwestern Louisiana
Lafayette, LA 70504

Abstract

Allocating submeshes with any arbitrary sizes in a mesh computer system efficiently is desirable. In this paper, we propose a new processor allocation strategy which can apply to any mesh system and recognize submeshes with arbitrary sizes at any locations in a mesh system. The proposed strategy allocates a submesh of exactly the size requested by an incoming task, completely avoiding internal fragmentation. Because of its efficient allocation, this strategy exhibits better performance than an earlier allocation strategy based on the buddy principle. An efficient implementation of this strategy is presented. Extensive simulation runs are carried out to collect experimental performance measures of interest under different allocation schemes for comparison.

1. Introduction

With advances in VLSI technology, extensive research has been carried out over the years to design various parallel architectures. Due to its simple and regular structure suitable for VLSI implementation, the mesh topology has been drawing considerable attention from researchers of different fields in recent years. Based on this topology, several prototypes and commercial systems have been built or under construction, such as ILLIAC IV [1], Goodyear Aerospace MPP [2], the Tera Computer System [3], and the Touchstone Delta System [4]. They exhibit a high potential for the parallel execution of various algorithms and may deliver as good performance as typical supercomputers at a much lower cost.

Considered in this paper is a mesh connected system where the resources are the processor nodes forming submeshes of various sizes. An incoming task incident on the system is analyzed for decomposability and the number of processors required for the task is determined; an appropriate submesh is then assigned to the task. The job of a processor allocator is to find an available submesh with a size just sufficient to meet the need of the task. In this paper, we limit our attention to the processor allocation of

two-dimensional (rectangular) mesh systems. An incoming task is assumed to need a rectangular submesh. Allocation in a high dimensional mesh system can be investigated similarly.

The recognition of various submeshes in a mesh system using the two-dimensional buddy strategy has been proposed recently by Li and Cheng [5]. The strategy is a generalization of the traditional one-dimensional binary buddy system developed for memory management [6]. This strategy, however, is applicable only to the square mesh system, and allocates only square submeshes, both with certain predefined sizes in the side. More specifically, the side lengths of the system and its allowable submeshes are limited to exact powers of 2. Consequently, this strategy can neither apply to square mesh systems with side lengths being not powers of 2 nor be incorporated in a *non-square* mesh system like the Touchstone Delta System (which is being built by an Intel division for a group of 14 U.S. research organizations) [4]. Besides, an incoming task is always assigned to a square submesh with the side length of 2^i such that 2^i is no less than the maximum of the two side lengths of the actually required submesh. As a result, this strategy often allocates a submesh with its sizes "dilated" and is thus termed a *dilated* submesh allocation scheme. Since it cannot recognize rectangular submeshes with arbitrary side lengths, this strategy tends to result in large *internal fragmentation*.

In light of the facts that processors in a mesh system are not always arranged in a square fashion and a dilated submesh allocation scheme often wastes a significant amount of processor resources and hence imposes unnecessary delay on an incoming task (due to large internal fragmentation), it is desirable to consider a more efficient allocation technique which can apply to any mesh system, recognize submeshes with arbitrary sizes, and assign a submesh of the precise size to an incoming task, totally eliminating internal fragmentation. In this paper, we propose a new strategy which achieves precise submesh allocation in *any* mesh system even with unequal side lengths and, thereby, high processor utilization as

This work was supported by the NSF under Grant MIP-8807761.

well as short task waiting time. The proposed strategy searches an available submesh capable of accommodating an incoming task by examining a sequence of candidate "frames" each with the same size (in both sides) as the requested submesh, until the processors within a "frame" are found all available and are assigned to the incoming task. Determining whether all processors within a frame are available or not can be done quickly by managing three simple sets appropriately (without checking every processor individually). This strategy can be implemented in a centralized or distributed manner. It is an efficient strategy for allocating precise submeshes on demand. Simulation results indicate that the proposed strategy gives rise to significantly improved performance over a dilated submesh allocation scheme, while maintaining time complexity involved reasonably low.

2. Nomenclature and Related Work

A two-dimensional (rectangular) mesh, denoted by $M_2(w, h)$, consists of $w \times h$ nodes arranged in a $w \times h$ two-dimensional grid. (Notice that a node refers to a processor; nodes and processors are used interchangeably.) The node in row i and column j is identified by address $\langle i, j \rangle$ and is connected through direct communication links to $\langle i \pm 1, j \rangle$ and $\langle i, j \pm 1 \rangle$, for $0 \leq i < w$ and $0 \leq j < h$ (a boundary node has fewer neighbors). A two-dimensional (rectangular) submesh in $M_2(w, h)$, denoted by $S_2(w', h')$, is a subgrid $M_2(w', h')$ such that $1 \leq w' \leq w$ and $1 \leq h' \leq h$. The address of a submesh is denoted by a quadruple (x, y, x', y') , where $\langle x, y \rangle$ indicates the lower left corner coordinate of the submesh, and $\langle x', y' \rangle$ is the upper right corner. Fig. 1 shows a rectangular mesh $M_2(5, 4)$. Nodes $\langle 2, 1 \rangle$, $\langle 3, 1 \rangle$, $\langle 4, 1 \rangle$, $\langle 2, 2 \rangle$, $\langle 3, 2 \rangle$, and $\langle 4, 2 \rangle$, for example, constitute a submesh $S_2(3, 2)$ with address $(2, 1, 4, 2)$.

The processors in $M_2(w, h)$ must be allocated to incoming tasks efficiently so as to reduce fragmentation and obtain high processor utilization. An incoming task T requesting a rectangular submesh $S_2(w', h')$, with $1 \leq w' \leq w$ and $1 \leq h' \leq h$, is specified by $T = (w', h')$. In what follows, we briefly review prior allocation strategies.

Two-Dimensional Buddy Strategy

The two-dimensional buddy strategy [5], being a generalization of the conventional one-dimensional binary buddy system [6], is applicable only to the situation where all incoming tasks need square submeshes $S_2(w', w')$'s, and the system itself is a square mesh $M_2(w, w)$, with both w' and w being exactly

powers of 2. Assuming $w = 2^r$, this strategy maintains a set of lists of available square submeshes with side length $w' = 2^{r'}$ such that one list is for an r' value, where $0 \leq r' \leq r$. An incoming request for a submesh of side $q (= 2^k)$ is allocated the first element in the list of free square submeshes $S_2(2^k, 2^k)$'s, if the list is not empty; otherwise, an available square submesh with $r' > k$ is decomposed before being allocated to the request. Although its allocation procedure is straightforward, this strategy involves a complicated deallocation procedure, whenever a submesh is released, in order to guarantee its correctness. Specifically, the deallocation procedure first has to merge the released submesh with other submeshes of an identical size, if any, to form a bigger submesh, and then remove the merged submeshes from the list. This process will be repeated for the newly formed submesh until nothing can be done further. To this end, the strategy is implemented [5] by assigning an *order* to each submesh in the list through which each submesh can identify its three buddies; a bigger submesh is created from a submesh together with its three buddies if the three buddies are all in the list.

This strategy cannot be incorporated even in square mesh systems with side lengths unequal to powers of 2, not to mention in a non-square mesh system like the Touchstone Delta System [4], where 528 processors are arranged in a two-dimensional mesh $M_2(33, 16)$. Under the strategy, an incoming request for $S_2(w', h')$ is assigned a square submesh $S_2(w, w)$, with w being the maximum of w' and h' rounded up to the nearest power of 2, i.e., $w = 2^{\lceil \log_2(\max(w', h')) \rceil}$ where $\max$ is a function that returns the bigger value of w' and h' . Consequently, large *internal fragmentation* tends to exist and a significant amount of processor resources could be wasted, which may otherwise form certain submeshes able to accommodate the subsequent incoming tasks. For example, this strategy allocates an $S_2(8, 8)$ to a request which actually needs a submesh $S_2(2, 5)$, wasting 54 processors. A task may be unnecessarily delayed to enter the system as a result of this inefficient submesh allocation.

3. Proposed Allocation Strategy

In the following, we introduce a new submesh allocation strategy based on frame sliding, termed the *FS strategy*, which can recognize and assigns submeshes with actual sizes needed by incoming tasks. The FS strategy completely eliminates internal fragmentation and exhibits better performance than the buddy strategy.

3.1. Basic Idea

For a "frame" of a given size, all processors within the "frame" can be told immediately when the location of a certain frame point, say the lower left corner of the frame, is specified. Hereafter, a "frame" is positioned by its lower left corner address, and a "frame" refers to those processors within the "frame."

We observe that in response to a request, the buddy strategy basically involves checking "frames" at certain specific locations dictated by the size of the requested submesh. To give an example, consider the mesh system $M_2(16, 16)$ illustrated in Fig. 2(a). When an incoming task $T = (8, 8)$ arrives, the buddy strategy checks only four locations, namely, A , C , I , and K , for availability. On the other hand, it examines locations A through P (but not other locations) in response to a task $T = (4, 4)$. Since all possible locations selected by the buddy strategy are fixed and implicit according to the size of a requested submesh, the strategy can be implemented in a way that a free list is designated for keeping track of available square submeshes of a certain size, and there is no need to explicitly record the locations of all allocated submeshes. Although simple at the allocation step, this scheme is fairly restricted in identifying the possible locations and sizes of recognizable submeshes, making it inefficient potentially. Consider Fig. 2(b), for instance, an incoming task of $T = (5, 4)$ cannot enter the mesh system in the presence of four allocated square submeshes with addresses $(0, 0, 1, 1)$, $(8, 0, 11, 3)$, $(0, 8, 7, 15)$ and $(8, 8, 15, 15)$, as shown in the figure, despite that a "frame" with size 5×4 at node $\langle 2, 0 \rangle$ involves only free processors and would be assigned if it were recognizable. Furthermore, the deallocation step of this strategy tends to be complex.

To remedy this inadequacy and improve flexibility in identifying available submeshes (both in their sizes and locations), we propose to use the idea of a "frame" whose size is identical to that of the requested submesh (rather than fixed and square) and whose possible location is dependent on the available resources at the time of allocation (rather than implicit and predefined). When processors in the currently examined "frame" are not all available, the "frame" is slid over the "plane" of the mesh system for searching next candidates, with the horizontal and vertical strides equivalent respectively to the width and height of the requested submesh. The search process starts from the "frame" at the lowest leftmost available processor. When the same incoming task $T = (5, 4)$ reaches Fig. 2(b), for example, a 5×4 "frame" at node $\langle 2, 0 \rangle$ is examined first. In this case, the first attempt succeeds and the processors in

the examined "frame" will be assigned to the task.

If the processors in an examined "frame" are not all available, the next candidate "frame" is checked, until a "frame" involving only free processors is found or all candidate "frames" are exhausted. In the latter case, no suitable submesh exists for the incoming task and the task cannot enter the mesh system at this time. Consider an incoming task $T = (8, 4)$ incident to the mesh system shown in Fig. 2(b). The first examined 8×4 "frame", which is again at node $\langle 2, 0 \rangle$, is unavailable and the second candidate "frame" at node $\langle 10, 0 \rangle$ (which is determined by sliding the "frame" rightwards along the horizontal direction with a stride 8, the width of the requested submesh) is then checked. This candidate apparently is not available since node $\langle 10, 0 \rangle$ itself is not free. The third candidate is not obtained by sliding the "frame" further rightwards, but by sliding the "frame" along the vertical direction with a stride 4, the height of the requested submesh, and then moving leftwards such that the right end of the "frame" is at the right boundary of the mesh system, reaching the candidate "frame" at node $\langle 8, 4 \rangle$. Now, this "frame" is available and all processors in it are assigned to the incoming task. Note that after making a vertical stride, the "frame" is slid repeatedly along the horizontal direction until it exceeds the boundary in the opposite side. In general, the search process is carried out by examining a sequence of "frames" which altogether cover the entire mesh "plane."

3.2. Implementing the FS Strategy

The proposed strategy can be implemented efficiently, without examining the availability of every processor in a candidate "frame" individually during search, by appropriately generating two sets according to the requested submesh size and the current occupancy situation in the mesh system. Some definitions are given before an efficient implementation of the FS strategy is presented.

Definition 1: The *base* of a (sub)mesh is the processor at the lower left corner of the (sub)mesh. The *busy set* of a mesh system is the collection of all current allocated submeshes in the system.

In Fig. 1, for example, processors $\langle 0, 0 \rangle$ and $\langle 2, 1 \rangle$ are respectively the bases of mesh $M_2(5, 4)$ and the submesh with address $(2, 1, 4, 2)$. The busy set of the mesh shown in Fig. 2(b) consists of four elements corresponding to the four allocated square submeshes, i.e., $\{(0, 0, 1, 1), (8, 0, 11, 3), (0, 8, 7, 15), (8, 8, 15, 15)\}$.

Definition 2: The *coverage* of an allocated submesh α with respect to an incoming task T , denoted

by $\xi_{\alpha,T}$, is a collection of processors each of which cannot serve as the base of any available submesh for accommodating T . The *coverage set* is the union of the coverages of all allocated submeshes, i.e., the elements in the busy set.

As an example, in Fig. 1, if the submesh with address $(2, 1, 4, 2)$ is allocated and $T = (2, 1)$ is requested by an incoming task, the coverage of the allocated submesh comprises all processors in the submesh $(1, 1, 4, 2)$. In general, let (x, y, x', y') be the address of an allocated submesh α , for an incoming task $T = (i, j)$, it is easy to obtain that coverage $\xi_{\alpha,T}$ comprises all the processors of the submesh $(x-i+1, y-j+1, x', y')$.

Definition 3: A *reject submesh* with respect to an incoming task is a submesh involving processors which can never be the base of any available submesh for accommodating the incoming task. The *reject set* is the collection of all such reject submeshes.

To give an example, the two submeshes $(3, 0, 4, 3)$ and $(0, 3, 4, 3)$ in Fig. 1 form the reject set with respect to an incoming task $T = (3, 2)$. In general, for system $M_2(w, h)$, the reject set with respect to an incoming task $T = (i, j)$ comprises two submeshes with addresses $(w-i+1, 0, w-1, h-1)$ and $(0, h-j+1, w-1, h-1)$.

The next theorem serves as the basis of an efficient implementation of our strategy.

Theorem 1: Let Ψ , Δ , and x be the coverage set, the reject set with respect to an incoming task, and the location of a frame, respectively, then the processors within the "frame" are all available only if x does not belong to any element in Ψ or Δ .

This theorem follows directly from the fact that (i) if x belongs to any element in Ψ , the frame would be overlapped with some allocated submesh(es), and (ii) if x belongs to any element in Δ , (a portion of) the frame would be outside the boundary of the mesh system. When an incoming task arrives, all candidate "frames" can be determined immediately and are examined in sequence, starting with the one positioned at the lowest leftmost available processor. Whether an examined "frame" is available or not is told quickly by checking its lower left corner node against the coverage set and the reject set, which are produced on receiving the incoming task. The proposed FS allocation strategy is formally described as follows.

(The busy set is empty initially.)

Processor Allocation:

Step 1. Set $i := w'$ and $j := h'$, where $T = (w', h')$ is the current incoming task.

Step 2. Generate the coverage set (from the busy set) and the reject set according to i and j .

Step 3. Find the lowest leftmost available processor $\langle x, y \rangle$. Examine the lower left corner node of every candidate "frame" in sequence, starting with $\langle x, y \rangle$. If the examined node is not free, proceed to the next candidate; else compare the node against the coverage set and the reject set. If it belongs to any element of the two sets, proceed to the next candidate; otherwise, go to the next step. If all candidates are exhausted, go to Step 5.

Step 4. Include the selected submesh (i.e., current "frame") in the busy set and then allocate the submesh to T . Stop.

Step 5. Attach T to the task queue and wait until a submesh is released.

Processor Relinquishment:

Remove the released submesh from the busy set.

Our allocation strategy can be carried out by either a host processor centrally or using multiple free processors in a distributed manner. In a centralized approach, the host processor keeps the busy set, and only the host processor is responsible for allocation. When a request arrives at the mesh system, the host processor executes the allocation steps provided above.

In a distributed approach, all free processors are involved in the job of processor allocation. Each free processor checks in parallel if it can be the base of a submesh suitable for the incoming task, but the busy set is managed uniquely by the host processor. Upon receiving a request, the host processor starts to generate the coverage set and the reject set. The union of the two generated sets is then broadcast to all free processors. After receiving broadcast information, a free processor starts to examine if it resides in any element of the received set concurrently and independently. Each involved processor sends the result back to the host processor, indicating whether it belongs to any element (i.e., a "P" report) or not (i.e., an "N" report). The host, on receiving an "N" report from an involved processor, immediately aborts all the unfinished checking processes at the other free processors and assigns the submesh with this processor as its base to the incoming task. The host processor also updates its busy set. On the other hand, when η ($=$ the total number of free processors) "P" reports are received, the host processor knows that there is no submesh available for the incoming task.

It should be noted that all processors involved in task allocation are set free at the end of allocation, and some of them may be part of the allocated

submesh. During the entire course of performing the allocation job, the involved processors need not communicate with one another; instead they communicate only with the host processor.

4. Experimental Performance Evaluation

Extensive simulation runs are conducted to collect the important performance measures of different strategies for mesh processor allocation. Simulation studies on various mesh sizes ranging from 16×16 to 256×256 were carried out, and all of the simulation results follow a similar trend to the results reported here for the $M_2(64, 64)$. For brevity, we shall not present all the other simulated results. (Recall that the two-dimensional buddy strategy¹, applies only to square mesh systems with predefined sizes in the side, so only the square meshes with each side length being a power of two are chosen for simulation.)

The simulation model is described first. Task allocation is carried out centrally by a dispatching processor exclusive from the simulated mesh system so that a request for all of the mesh processors is allowed. Initially, the entire simulated mesh is free, and 1000 tasks are generated and queued at the dispatcher. The side lengths of submeshes requested by these 1000 tasks are assumed to follow a given distribution. The residence times of allocated submeshes are assumed to be uniformly distributed between 5 and 10 time units. The dispatcher attempts to allocate the task at the top of the queue first. If it fails to identify an available submesh of the desired size for the task, the dispatcher reattempts when a submesh is released until an available submesh is eventually obtained, i.e., the FCFS scheduling discipline is followed. When an available submesh is allocated to the task, the task is removed from the queue and the next task is served at the following time unit. No new task is generated during the course of simulation.

Under the above simulation model, we collected such performance measures as the completion time (Ξ — the time taken to finish all the 1000 tasks), processor utilization (the percentage of a processor being utilized per unit time) over the period Ξ , internal fragmentation (F_{int}), external fragmentation (F_{ext}), and total fragmentation (F_{tot}). The last three fragmentation measures are similar to what are defined in [5]: F_{int} is the percentage of overallocated mesh

processors (i.e., those exceeding what are actually needed) over allocated mesh processors, F_{ext} is the percentage of total available mesh processors over the total mesh processors at each allocation failure, and $F_{tot} = (1 - F_{ext}) \times F_{int} + F_{ext} = F_{int} + F_{ext} - F_{int} \times F_{ext}$. The simulation results are averaged over five independent runs.

The results for processor allocation in $M_2(64, 64)$ are shown in Table I. The side lengths of submeshes requested by the 1000 tasks are governed by four distributions — the uniform, exponential, decreasing and increasing distributions. Note that the two side lengths of each requested submesh are under the same distribution, but two different random number streams, one for each side, are employed to generate the length value. A rectangular submesh so generated can be with any arbitrary size.

The uniform distribution indicates that each side length is uniformly distributed between 1 and 64, which seems to reflect well the situations where the nature of tasks to be executed is unknown. By contrast, the decreasing distribution indicates that the probability of requesting a larger submesh is lower, which may reflect better the cases when most tasks have reasonable parallelism but are not suitable for fine-grain partition. The probabilities of side lengths of requested submeshes under the decreasing and increasing distributions are shown below the table. $P_{[a, b]}$ denotes the probability of side length to fall within $Q = [a, b]$ and all possible side lengths in Q are uniformly distributed. The mean of the exponential distribution is one half of the mesh system side size, 32.

The results shown in the table are reasonably accurate. Given 95% confidence, for instance, the first value 7099.2 (in time unit) provided in the table has the calculated confidence interval half-width over the five replications equal to 87.8, meaning we are 95% confident that the true result would fall into the interval 7099.2 ± 87.8 , or equivalently, $7099.2 \pm 1.24\%$. This simulated value has less than 2% error.

It can be observed from the table that the FS strategy has significantly less completion time and notably larger processor utilization when compared with the buddy strategy, particularly under the uniform, exponential and decreasing distributions. Shorter completion time is a direct result of more efficient processor utilization using the FS strategy, which totally eliminates internal fragmentation (whose value ranges from roughly 47% to 81%, as experienced by the buddy strategy shown in the table). This clearly demonstrates the advantage of our strategy over a dilated submesh allocation scheme, like the buddy strategy. Notice that the FS

¹The simulation program for the buddy strategy was provided by K. Li and K. H. Cheng.

strategy gives rise to larger external fragmentation under every distribution, as listed in the table. This is probably because the variety of submesh sizes grows substantially using the FS strategy, and thus the mesh system tends to be partitioned into more different sized blocks, making an incoming task less likely to be fit exactly in a free block. Nevertheless, total fragmentation is always far less for the FS strategy.

It is also interesting to observe that the improvement in completion time and processor utilization under the increasing distribution is least pronounced among the four distributions. This could be due to the fact that under the increasing distribution, the probability of requesting a large submesh increases, and an appropriate free submesh is less likely to be immediately available. It often takes many deallocation events to make enough room for the task requesting a large submesh. As a result, the FS strategy under this distribution does not experience as drastic performance improvements as under the other three distributions.

The last two rows give time complexity using the FS strategy, measured by the expected number of comparisons per (successful) allocation attempt. By a comparison, we mean taking one available processor to see if it is in one element of the coverage set (or the reject set). A comparison thus involves comparing the two coordinates of the processor address with the two ranges specified by the element. Since every attempt often requires a small number of comparisons and each comparison includes only simple operations, the mean search time for finding an available submesh is very short, yielding low time complexity. In the worst case, it takes about 70 comparisons for an allocation attempt, which could still compare favorably with the buddy strategy because the buddy strategy tends to involve high overhead in the deallocation procedure. If a desired submesh is available (a successful attempt), it takes a less mean search time to identify such a submesh. The worst case happens under a decreasing distribution because tasks then request smaller submeshes with a higher probability and at any time, more allocated submeshes (most of which are small) exist simultaneously in the mesh system, leading to more elements in the coverage set that must be compared at every allocation attempt.

The relationship between fragmentation and the mean side length of submeshes is also studied. In this study, incoming tasks are assumed to request only square submeshes (so the size of a requested submesh can be specified by only one side length). The uniform and exponential distributions are chosen for consideration here because their variances can easily

be defined for every mean side length value of interest. The simulation results for $M_2(64, 64)$ are plotted in Figs. 3-5, where the variance of side length for the uniform distribution equals $\sigma^2 = 12^2$. Fig. 3 illustrates internal fragmentation as a function of mean side length for the buddy strategy (recall that the FS strategy has nil internal fragmentation). As shown in the figure, internal fragmentation stays consistently above 45% throughout the entire mean side length range under study for both distributions. In Fig. 4, external fragmentation is found larger in the FS strategy than in the buddy strategy, as might be expected. While the external fragmentation gap between the two strategies stays virtually unchanged over the range of simulated mean side length under the exponential distribution, the gap gradually increases as the mean side length grows under the uniform distribution. As far as total fragmentation is concerned, the FS strategy always exhibits smaller values for both distributions, as depicted in Fig. 5. The reduction in total fragmentation due to the FS strategy is significant, indicating that the FS strategy has a clear performance gain.

5. Alternative Allocation Policy

As has been demonstrated in the previous section, the FS strategy has a relatively larger external fragmentation than the buddy strategy. In the following, we explore an approach for reducing external fragmentation. Instead of allowing an arbitrary sized submesh to be allocated as the prior FS strategy, we assume that an incoming request for $T = (w', h')$ is assigned a square submesh $S_2(w, w)$, with w being the larger of w' and h' rounded up to the nearest power of 2, i.e., $w = 2^{\lceil \log_2(\max(w', h')) \rceil}$. This means that the total number of allowable submesh sizes now is exactly the same as that of the buddy strategy. However, the sequence of examining candidate "frames" remains unchanged in this alternative FS strategy, and thus a candidate "frame" is again positioned according to current occupancy in the mesh system and is not confined to fixed, predefined locations, as the buddy strategy. It is apparent that internal fragmentation now would become large, just like what is experienced by the buddy strategy.

The simulation results under this allocation policy for $M_2(64, 64)$ are listed in the row of FS_A in Table II. Here, side length is governed by the uniform distribution, ranging uniformly from 1 to 64. The next row in the table provides the results of the prior FS strategy, which allocates a precise submesh to a request (i.e., the same results as given in the column of "uniform" in Table I). As might be expected,

external fragmentation now reduces to below 10% but internal fragmentation rises to above 68%, leading to a big net increase in total fragmentation and thus lower processor utilization. The FS strategy using the precise allocation policy enjoys superior overall performance and is a better choice, despite involving slightly more time overhead.

It is interesting to compare the performance results of this alternative FS strategy and the results of the buddy strategy (also listed in the column of "uniform" of Table I). Surprisingly, the two strategies now display exactly identical results, implying that the proposed FS strategy behaves virtually the same as the buddy strategy when it has an equal number of allowable submesh sizes. Under this situation, the FS strategy is very fast, taking on an average approximately two comparisons per successful allocation attempt. Note that the mean number of comparisons per allocation attempt is less than one in the alternative FS strategy because in many cases, the mesh system is totally occupied (since the whole mesh system is always allocated to any request of a submesh with one side length > 32) and no comparison is needed at all to know that no available submesh is then existing.

To compare the costs of this alternative FS strategy and the buddy strategy, the total number of operations performed to finish the allocations and deallocations of the 1000 tasks in $M_2(64, 64)$ are accumulated and listed in Table III for both strategies. By an operation, we mean a comparison (described in the previous section) for the alternative FS strategy, whereas for the buddy strategy, we mean decomposing a bigger available submesh, inserting a submesh in the appropriate position of a particular list, or merging and removing a submesh together with its three buddies from the list. We concern only the major operations involved in the strategies, leaving out trivial operations such as calculating the order in the buddy strategy, generating the coverage in the FS strategy for a particular submesh, and so on. The side lengths of submeshes requested by the 1000 tasks are under the same four distributions as given in Table I. It can be observed from Table III that the alternative FS strategy involves more operations (by 20% to 58% more) under three distributions but requires slightly fewer operations under the uniform distribution. However, it should be noted that an operation of the alternative FS strategy is much simpler than an operation of the buddy strategy. We thus believe that the total time spent could be more in the buddy strategy than in the alternative FS strategy.

6. Conclusions

In this paper, we have introduced an efficient mesh processor allocation strategy which can apply to any mesh system and recognize submeshes with arbitrary sizes at any locations. The proposed strategy allocates a submesh of the precise size to an incoming task, completely eliminating internal fragmentation. As a result, this strategy leads to far better processor utilization and substantially reduces total time spent in finishing a batch of tasks, when compared with an earlier allocation strategy based on the buddy principle. An effective implementation of the strategy is also presented, which greatly simplifies the search of candidate "frames" for an available submesh. Simulation results confirm that our proposed strategy is consistently superior when allocating submeshes whose side lengths follow any of the four distributions, in terms of performance measures of interest. The mean search time per allocation attempt is fairly short, involving at most tens of simple comparisons.

One way which reduces the external fragmentation of our strategy is simulated and discussed. Our strategy is found to behave virtually the same as the buddy strategy if its allowable submesh sizes are limited to those permitted by the buddy strategy. It is observed from simulation that total time spent in allocating 1000 tasks could be less using the alternative FS strategy than using the buddy strategy.

References

- [1] G. H. Barnes *et al.*, "The Illiac IV Computer," *IEEE Trans. on Computers*, vol. C-27, No. 1, pp. 84-87, Jan. 1968.
- [2] K. E. Batcher, "Bit-Serial Parallel Processing Systems," *IEEE Trans. on Computers*, vol. C-31, No. 5, pp. 377-384, May. 1982.
- [3] R. Alverson *et al.*, "The Tera Computer System," *Proc. 1990 Intl. Conf. on Supercomputing*, June 1990, pp. 1-6.
- [4] G. Zorpette, "Technology 1991: Minis and Mainframes," *IEEE Spectrum*, pp. 40-43, Jan. 1991.
- [5] K. Li and K. H. Cheng, "A Two Dimensional Buddy System for Dynamic Resource Allocation in a Partitionable Mesh Connected System," *Proc. ACM Computer Science Conference*, Feb. 1990, pp. 22-28 (also to appear in *IEEE Trans. Parallel and Distributed Systems*).
- [6] J. L. Peterson and T. A. Norman, "Buddy Systems," *Communications of the ACM*, vol. 20, No. 6, pp. 421-431, June 1977.

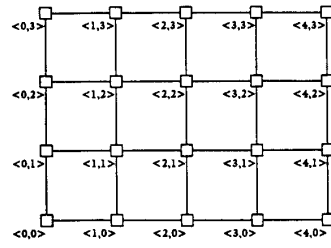


Fig. 1. A rectangular mesh $M_2(5, 4)$.

Table I. Simulation results of performance measures for $M_2(64, 64)$

Performance measure		Submesh side length distribution			
		uniform	exponential	decreasing*	increasing**
Completion time	Buddy	7099.2	5697.5	5011.8	7489.7
	FS	4191.7	2445.5	1898.5	6450.1
Processor utilization	Buddy	26.6	16.1	12.7	51.8
	FS	45.2	37.5	33.5	60.1
Internal fragmentation	Buddy	68.2	77.9	81.2	46.8
	FS	0.0	0.0	0.0	0.0
External fragmentation	Buddy	9.5	17.7	21.5	1.5
	FS	34.6	35.4	34.4	28.9
Total fragmentation	Buddy	71.2	81.8	85.2	47.6
	FS	34.6	35.4	34.4	28.9
FS time	NC***	6.7	26.6	52.0	2.8
Complexity	NC****	8.6	31.8	69.3	3.3

* $P_{[1, 8]} = 0.4$, $P_{[9, 16]} = 0.2$, $P_{[17, 32]} = 0.2$, $P_{[33, 64]} = 0.2$.
 ** $P_{[1, 32]} = 0.2$, $P_{[33, 48]} = 0.2$, $P_{[49, 64]} = 0.2$, $P_{[57, 64]} = 0.4$.
 ***number of comparisons per successful attempt.
 ****number of comparisons per attempt.

Table II. Performance comparison of different allocation policies

Performance measure	Completion time	Processor utilization	Internal fragment.	External fragment.	Total fragment.	NC*	NC**
FS _A	7099.2	26.6	68.2	9.5	71.2	2.1	0.8
FS	4191.7	45.2	0.0	34.6	34.6	6.7	8.6

*number of comparisons per successful attempt.
 **number of comparisons per attempt.

Table III. Number of operations involved for $M_2(64, 64)$

Strategy	Submesh side length distribution			
	uniform	exponential	decreasing	increasing
Buddy	2632	3824	4503	1295
FS _A	2456	4671	5387	2044

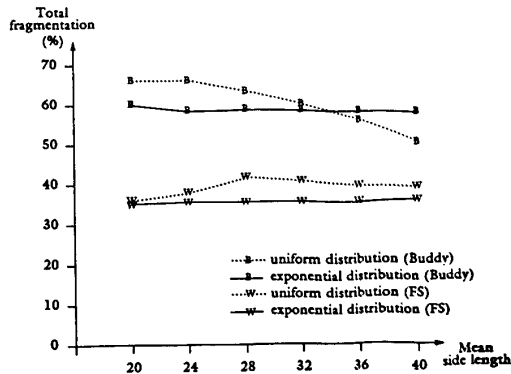


Fig. 5. Total fragmentation of the buddy and the FS strategies.

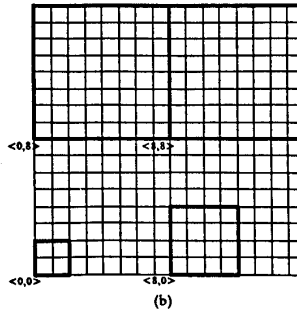
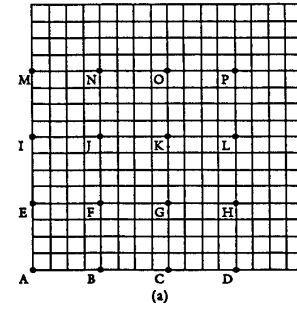


Fig. 2. (a) Recognizable submeshes in $M_2(16, 16)$ under the buddy strategy.

(b) Four allocated square submeshes present in $M_2(16, 16)$, indicated by bold boxes.

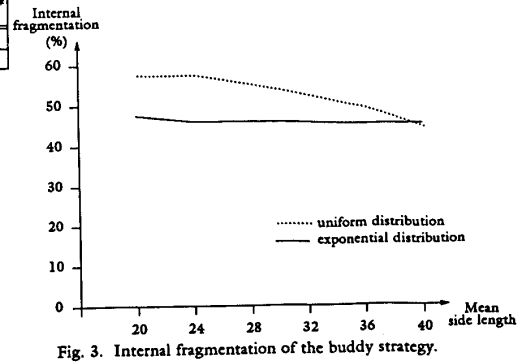


Fig. 3. Internal fragmentation of the buddy strategy.

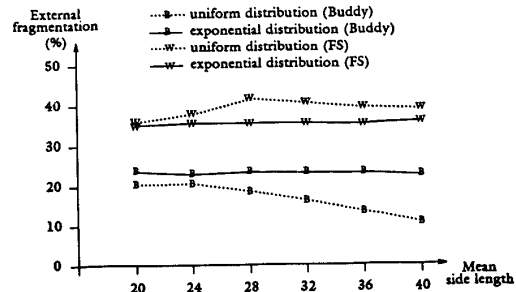


Fig. 4. External fragmentation of the buddy and the FS strategies.